Avaliação Técnica de QA – Vaga: Analista de QA Sênior

Candidato(a): _____
Data de Aplicação: ____/____/_____
Nível da Vaga: Analista de QA Sênior

Instruções gerais

- Esta avaliação visa medir a capacidade analítica, visão arquitetural, qualidade técnica de código (Web e Mobile), engenharia de performance e aplicação estratégica de IA.
- O sistema base para os exercícios é o e-commerce **https://www.saucedemo.com/**.
- Como esta é uma posição para **nível Sênior**, **os cenários de teste não estão pré-definidos**. Esperamos que você explore a plataforma, priorize os fluxos críticos de negócio, identifique riscos e justifique suas escolhas técnicas.
- Uso da IA no decorrer do teste é liberado.

1. Automação web com Cypress e levantamento de cenários

Acesse o e-commerce **https://www.saucedemo.com/**, analise a aplicação como um todo e defina a sua estratégia de testes E2E.

O que você deve entregar:

1. **Mapeamento e Priorização de Cenários:**
    - Liste os cenários que você considerou essenciais para automação Web em um e-commerce.

    - Justifique a priorização com base em risco de negócio (RBT - *Risk-Based Testing*), impacto financeiro e experiência do usuário.

2. **Implementação em Cypress:**
    - Implemente os cenários priorizados utilizando **Cypress**.

    - Esperamos um código de nível sênior: organização arquitetural (Page Objects, App Actions ou Custom Commands), seletores desacoplados, tratamento adequado de asserções, boas práticas contra flaky tests e separação de massa de teste.

Prompt — Análise RBT e Planejamento de Automação Web | SauceDemo

.PROMPT DEFINITIVO — Automação Web E-commerce | Cypress + TypeScript | SauceDemo

1. Papel (Role)

Atue como um **QA Sênior / Quality Engineer / SDET**, especialista em:

- Automação de testes web com Cypress e TypeScript.
- Arquitetura de automação: Page Objects, App Actions e Custom Commands.
- Risk-Based Testing (RBT).
- Técnicas de teste ISTQB.
- Testes funcionais, de integração e end-to-end.
- Testes de API utilizando `cy.request()` e `cy.intercept()`.
- Seletores estáveis e desacoplados.
- Massa de teste, fixtures e tipagem TypeScript.
- Prevenção, investigação e tratamento de flaky tests.
- CI/CD, observabilidade de testes e análise de falhas.
- IA generativa aplicada à engenharia de qualidade.

Sua responsabilidade é **analisar a aplicação, definir uma estratégia de automação baseada em riscos e implementar um projeto completo e sustentável**.

Não entregue apenas exemplos de código, uma estrutura de pastas vazia ou um único teste de demonstração.

2. Contexto (Context)

Aplicação: https://www.saucedemo.com/

Tipo: e-commerce demonstrativo.

Framework: Cypress.

Linguagem: TypeScript.

IDE: Visual Studio Code.

Objetivo: desenvolver uma suíte de automação web para os principais fluxos de negócio do SauceDemo, abrangendo autenticação, catálogo, ordenação, carrinho, checkout, valores financeiros simulados e navegação.

O projeto deve seguir práticas compatíveis com uma atuação de QA Sênior, priorizando:

- Cobertura baseada em riscos.
- Legibilidade e manutenção.
- Reutilização sem abstração excessiva.
- Assertions relevantes.
- Testes independentes.
- Estabilidade das execuções.
- Rastreabilidade entre requisitos, riscos e casos de teste.

3. Instruções (Instruction)

Etapa 1 — Exploração completa da aplicação

Acesse https://www.saucedemo.com/ e investigue seus principais módulos:

1. Autenticação e logout.
2. Catálogo de produtos.
3. Detalhes dos produtos.
4. Ordenação por nome e preço.
5. Carrinho de compras.
6. Checkout e validação dos campos.
7. Resumo financeiro.
8. Finalização e confirmação.
9. Navegação e preservação de estado.
10. Comportamentos dos usuários especiais de demonstração.

Para cada funcionalidade, registre:

- Comportamento observado.
- Regras identificadas.
- Dados necessários.
- Dependências.
- Cenários positivos, negativos e alternativos.
- Possíveis riscos.
- Evidências.
- Dúvidas e limitações.

Não invente funcionalidades, regras de negócio ou contratos de API.

Se não conseguir explorar a aplicação interativamente, informe essa limitação e diferencie

informações documentadas de comportamentos realmente verificados.

Etapa 2 — Inspeção dos elementos com DevTools

Abra o SauceDemo em um navegador compatível e utilize as Ferramentas do Desenvolvedor para inspecionar os elementos necessários à automação.

Registre:

- Tag HTML.
- ID.
- Atributos `data-test`, `data-testid` ou equivalentes.
- Atributos de acessibilidade.
- CSS selector recomendado.
- Estratégia de localização.
- Evidência da inspeção.
- Status de confirmação.

Priorize seletores destinados a testes, como `data-test`.

Evite:

- Seletores baseados em posição.
- Classes CSS dinâmicas.
- XPath excessivamente complexo.
- Seletores fortemente dependentes da estrutura do DOM.
- Localizadores inventados.

Não apresente seletores retirados de documentação externa como se tivessem sido inspecionados diretamente.

Etapa 3 — Priorização por Risk-Based Testing

Avalie os cenários considerando:

- Risco de negócio.
- Impacto financeiro potencial.
- Impacto na conversão.
- Experiência do usuário.
- Probabilidade de falha.
- Criticidade do fluxo.
- Frequência de utilização.
- Estabilidade da automação.
- Custo de implementação e manutenção.

Utilize:

Score RBT = Probabilidade × Impacto

Atribua valores de 1 a 5.

Classifique inicialmente:

- **P0 — Crítico:** 16 a 25.
- **P1 — Alto:** 9 a 15.
- **P2 — Médio/Baixo:** 1 a 8.

Justifique as estimativas. Quando não existir histórico de defeitos ou mudanças, sinalize a incerteza.

A prioridade pode ser elevada justificadamente quando houver risco crítico de integridade financeira, segurança ou bloqueio total da compra.

Diferencie:

- Prioridade de negócio.
- Prioridade de automação.
- Severidade de um defeito.
- Ordem de implementação.

Não utilize esses conceitos como sinônimos.

Etapa 4 — Plano de testes

Mapeie os cenários utilizando técnicas apropriadas:

- Particionamento de equivalência.
- Análise de valor-limite.
- Tabela de decisão.
- Transição de estados.
- Testes baseados em casos de uso.
- Error guessing.
- Testes exploratórios baseados em risco.

Para cada cenário, documente:

- ID.
- Funcionalidade.
- Objetivo.
- Pré-condições.
- Massa de teste.
- Passos essenciais.
- Resultado esperado.
- Assertions.
- Técnica aplicada.
- Risco associado.
- Prioridade RBT.
- Classificação como smoke, regressão ou exploratório.

- Recomendação de automação.

Evite testes redundantes ou sem resultados verificáveis.

Etapa 5 — Cenários obrigatórios de referência

Considere os seguintes 24 cenários como backlog inicial. Confirme sua aplicabilidade antes da implementação.

Autenticação — `login.cy.ts`

- TC01 — Login com usuário válido.
- TC02 — Usuário bloqueado não consegue autenticar.
- TC03 — Tentativa de acessar página protegida sem autenticação.
- TC15 — Login com credenciais inválidas.
- TC16 — Logout e restrição de acesso após encerramento de sessão.

Catálogo e ordenação

- TC17 — Ordenação por preço crescente.
- TC18 — Ordenação por preço decrescente.
- TC19 — Consistência entre catálogo e detalhes do produto.
- TC21 — Ordenação por nome A–Z e Z–A.

Carrinho

- TC04 — Adicionar produto ao carrinho.
- TC05 — Adicionar múltiplos produtos.
- TC10 — Remover produto do carrinho.
- TC11 — Preservar itens durante navegação.

Checkout

- TC06 — Concluir compra com um produto.
- TC07 — Concluir compra com múltiplos produtos.
- TC08 — Validar subtotal, imposto e total.
- TC09 — Validar identidade e quantidade dos produtos no resumo.
- TC20 — Cancelar checkout e validar o estado do carrinho.

Validações do checkout

- TC12 — Primeiro nome obrigatório.
- TC13 — Sobrenome obrigatório.
- TC14 — Código postal obrigatório.

Cenários complementares e exploratórios

- TC22 — Integridade visual e imagens.
- TC23 — Comportamento de lentidão com usuário especial.

- TC24 — Investigar checkout com carrinho vazio.

Não implemente assertions definitivas para os cenários exploratórios sem critérios de aceite confirmados.

Caso algum cenário não seja automatizável de forma confiável, registre seu status e a justificativa.

Etapa 6 — Arquitetura de automação

Avalie tecnicamente as alternativas:

Page Objects

- Encapsulamento por página.
- Centralização de elementos e ações.
- Possibilidade de maior estrutura e manutenção.

App Actions

- Funções reutilizáveis por operação de negócio.
- Menor necessidade de classes.
- Composição simples de fluxos.

Custom Commands

- Comandos globais reutilizáveis.
- Conveniência para login, autenticação e preparação recorrente.
- Possível risco de ocultar comportamentos em comandos excessivamente grandes.

Arquitetura preferencial: híbrida, utilizando **App Actions + seletores centralizados + Custom Commands pontuais**.

Use Page Objects somente se houver uma justificativa técnica para adotá-los.

Não implemente os três padrões simultaneamente apenas para demonstrar conhecimento.

Justifique a arquitetura escolhida considerando legibilidade, desacoplamento, crescimento da suíte, custo de manutenção e facilidade de diagnóstico.

Etapa 7 — Estrutura obrigatória do projeto

Gere a seguinte estrutura:

saucedemo-cypress/

│

```
├── cypress/
│   ├── e2e/
│   │   ├── auth/
│   │   │   └── login.cy.ts
│   │   ├── catalog/
│   │   │   ├── products.cy.ts
│   │   │   ├── sorting.cy.ts
│   │   │   └── performance.cy.ts
│   │   ├── cart/
│   │   │   └── cart.cy.ts
│   │   └── checkout/
│   │       ├── checkout.cy.ts
│   │       └── validation.cy.ts
│   │
│   ├── actions/
│   │   ├── auth.actions.ts
│   │   ├── catalog.actions.ts
│   │   ├── cart.actions.ts
│   │   └── checkout.actions.ts
│   │
│   ├── selectors/
│   │   └── index.ts
│   │
│   ├── fixtures/
│   │   ├── users.json
```

```
│   │   ├── products.json
│   │   └── checkout.json
│   │
│   └── support/
│       ├── commands.ts
│       └── e2e.ts
│
├── src/
│   └── types/
│       ├── product.types.ts
│       └── checkout.types.ts
│
├── cypress.config.ts
├── cypress.d.ts
├── tsconfig.json
├── package.json
├── README.md
└── .gitignore
```

Todos os arquivos devem conter implementação ou documentação pertinente. Não crie arquivos vazios apenas para completar a árvore.

Caso sejam necessários arquivos adicionais, crie-os e justifique.

Etapa 8 — Implementação dos seletores

Centralize os localizadores em:

`cypress/selectors/index.ts`

Organize por domínio:

- Login.

- Catálogo.
- Produtos.
- Carrinho.
- Checkout.
- Menu e navegação.

Utilize nomenclatura clara.

Para elementos dinâmicos, crie funções parametrizadas com identificadores estáveis confirmados.

Não espalhe strings de seletores pelos Test Cases.

Caso o seletor não tenha sido confirmado, registre a pendência e não o apresente como definitivo.

Etapa 9 — Implementação das App Actions

Implemente todas as funções utilizadas pelos testes.

Exemplos de responsabilidades:

auth.actions.ts

- Login.
- Logout.
- Preparação de sessão.

catalog.actions.ts

- Selecionar produto.
- Ordenar catálogo.
- Navegar para detalhes.

cart.actions.ts

- Adicionar produto.
- Remover produto.
- Abrir carrinho.

checkout.actions.ts

- Iniciar checkout.
- Preencher dados.
- Continuar.
- Cancelar.
- Finalizar.

Utilize tipagem TypeScript para parâmetros e valores retornados.

Não utilize `any` indiscriminadamente.

Não retorne valores síncronos diretamente de comandos Cypress assíncronos.

Quando uma função precisar expor dados obtidos por Cypress, utilize o encadeamento apropriado e tipos como `Cypress.Chainable<T>`.

Mantenha as actions focadas em operações. Assertions de negócio devem permanecer preferencialmente próximas dos testes ou em verificações explicitamente nomeadas.

Etapa 10 — Implementação dos Custom Commands

Crie Custom Commands apenas para operações compartilhadas e justificadas.

Por exemplo:

```
cy.loginAsStandardUser()
```

Implemente também a declaração TypeScript correspondente em `cypress.d.ts`.

Verifique a compatibilidade com a versão instalada do Cypress.

Não adicione Custom Commands duplicando desnecessariamente a funcionalidade das App Actions.

Credenciais: utilize fixtures apenas para credenciais públicas de demonstração. Para segredos reais, utilize mecanismos seguros de configuração ou variáveis de ambiente.

Evite dependência de variáveis como `Cypress.env('SAUCE_PASSWORD')` sem configurar e documentar sua origem.

Etapa 11 — Separação de massa de teste

Utilize fixtures para dados estáticos e reutilizáveis:

users.json

- Usuário padrão.
- Usuário bloqueado.
- Credenciais inválidas.
- Mensagens de login esperadas.

products.json

- Identificadores dos produtos.
- Nomes.
- Preços confirmados.

- Informações relevantes.

checkout.json

- Dados válidos de comprador.
- Dados inválidos.
- Mensagens de campos obrigatórios.
- Valores esperados fundamentados em requisitos.

Diferencie dados de entrada e oráculos de resultado.

Evite depender de um preço lido da própria interface para comprovar que esse mesmo preço está correto.

Etapa 12 — Assertions e integridade das validações

Cada teste deve possuir assertions capazes de comprovar sua regra de negócio.

Priorize:

- Conteúdo esperado.
- Quantidade de elementos.
- Identidade do produto.
- Estado do carrinho.
- URLs e navegação.
- Mensagens específicas.
- Valores financeiros.
- Estado de autenticação.
- Confirmação de compra.

Não substitua validações funcionais por verificações genéricas como:

```
should('exist')
```

Utilize essas verificações somente quando a existência for efetivamente o resultado esperado.

Para valores monetários:

- Utilize uma representação numérica adequada, preferencialmente centavos inteiros quando aplicável.
- Defina regras de arredondamento.
- Calcule valores esperados independentemente.
- Compare subtotal, imposto e total.
- Não aceite diferenças de centavos sem regra documentada.

Etapa 13 — Prevenção e investigação de flaky tests

Aplique:

- Esperas condicionais com retry automático do Cypress.
- Seletores estáveis.
- Testes independentes.
- Preparação controlada dos dados.
- `beforeEach()` somente quando houver justificativa.
- Limpeza de estado entre testes.
- Uso apropriado de `cy.session()`, quando suportado e seguro.
- `cy.intercept()` somente para requisições realmente verificadas.
- Ausência de dependência da ordem de execução.
- Evidências de falha com screenshots e logs.

Evite:

- `cy.wait(5000)` como sincronização padrão.
- Seletores baseados em posições frágeis.
- `force: true` indiscriminadamente.
- Retries para esconder defeitos.
- Assertions realizadas antes da atualização observável da aplicação.
- Reutilização de estado inconsistente.

Quando um teste falhar, diferencie hipóteses de:

- Defeito de produto.
- Problema de automação.
- Falha de ambiente.
- Instabilidade de rede.
- Dados de teste inadequados.
- Problema de sincronização.

Etapa 14 — API e integração

Avalie se a aplicação disponibiliza endpoints HTTP observáveis e adequados para automação.

Quando existirem e houver autorização:

- Utilize `cy.request()` para preparar ou validar dados.
- Utilize `cy.intercept()` para observar e sincronizar operações.
- Valide status, conteúdo e consistência.
- Inclua cenários negativos pertinentes.

Não invente endpoints como `/api/products` apenas porque são comuns em e-commerce.

Se não existir API adequada e documentada, priorize a validação pela interface e registre essa limitação.

Etapa 15 — Testes parametrizados

Identifique cenários que podem compartilhar a mesma estrutura de teste com diferentes

massas de dados.

Exemplos:

- Credenciais inválidas.
- Campos obrigatórios.
- Ordenações.
- Seleção de produtos.

Utilize `forEach()` ou outra abordagem apropriada para gerar testes independentes e identificáveis.

Não crie um único teste enorme para diversas regras distintas.

Etapa 16 — Configuração do projeto

Implemente e revise:

package.json

- Dependências.
- Scripts de execução.
- Comandos para smoke e regressão.

cypress.config.ts

- `baseUrl`.
- `testIsolation`.
- Viewport.
- Timeouts.
- Configuração de screenshots.
- Configuração de vídeos, quando aplicável.
- Padrão de arquivos de teste.

tsconfig.json

- Tipagem Cypress.
- Suporte aos testes TypeScript.
- Configurações necessárias para JSON imports.
- Compatibilidade de módulos.

cypress.d.ts

- Tipos dos Custom Commands.

.gitignore

- `node_modules`.
- Logs.

- Screenshots e vídeos gerados.
- Arquivos locais sensíveis.
- Artefatos temporários.

Etapa 17 — Execução e qualidade do código

Após implementar o projeto:

1. Verifique as dependências.
2. Execute a validação TypeScript.
3. Verifique imports.
4. Verifique arquivos referenciados.
5. Verifique Custom Commands.
6. Verifique fixtures.
7. Procure seletores não confirmados.
8. Identifique assertions fracas.
9. Detecte cenários duplicados.
10. Verifique independência dos testes.

Quando o ambiente permitir, execute:

npm install

npx tsc --noEmit

npx cypress run

Ajuste o comando de TypeScript à estrutura efetiva do projeto.

Execute também as suítes smoke e regressiva separadamente, caso estejam configuradas.

Não declare testes aprovados sem execução real.

Caso a aplicação ou o ambiente não esteja acessível, diferencie:

- Código gerado.
- Código validado estaticamente.
- Teste executado.
- Teste aprovado.
- Teste pendente.

Etapa 18 — Métricas de qualidade do plano e da automação

Calcule e apresente, quando houver evidências suficientes:

Cobertura de requisitos

Quantidade de requisitos ou regras relevantes com testes associados dividida pelo total de

requisitos identificados.

Taxa de duplicidade

Quantidade de cenários redundantes dividida pelo total de cenários analisados.

Aderência à especificação

Quantidade de cenários sustentados por requisitos ou evidências dividida pelo total de cenários gerados.

Cenários críticos omitidos

Quantidade de riscos críticos identificados sem cenário correspondente.

Implementação

Cenários implementados divididos pelos cenários planejados para automação.

Execução

Cenários executados divididos pelos cenários elegíveis.

Aprovação

Cenários aprovados divididos pelos cenários executados.

Apresente numeradores, denominadores, fórmulas e limitações.

Não afirme 100% de cobertura sem um inventário completo e rastreável.

Etapa 19 — Geração dos arquivos e entrega

Gere fisicamente todos os arquivos necessários ao projeto.

Não entregue apenas uma arquitetura conceitual ou trechos isolados.

Crie um pacote:

`saucedemo-cypress-completo.zip`

O ZIP deve conter:

- Todos os Test Cases.
- Todas as App Actions.
- Seletores centralizados.
- Custom Commands.
- Fixtures.
- Tipos TypeScript.
- Configurações.

- README.
- Gitignore.
- Documentação de limitações e pendências.

O projeto precisa poder ser extraído e aberto diretamente no VS Code.

Também disponibilize o README separadamente.

Etapa 20 — Documentação e apresentação técnica

No `README.md`, inclua:

- Objetivo da automação.
- Estratégia RBT.
- Arquitetura escolhida e justificativa.
- Estrutura de diretórios.
- Instalação no Windows.
- Configuração do VS Code.
- Dependências.
- Como executar um teste específico.
- Como executar smoke e regressão.
- Como adicionar novos Test Cases.
- Como criar App Actions.
- Quando criar Custom Commands.
- Como utilizar fixtures.
- Como manter seletores.
- Como investigar falhas.
- Limitações e pendências.

Inclua uma seção final:

"Como defender esta arquitetura em uma entrevista de QA Sênior"

Explique as principais escolhas técnicas, trade-offs e decisões de qualidade.

4. Restrições (Constraints)

- Não inventar requisitos.
- Não inventar seletores.
- Não inventar endpoints.
- Não declarar inspeção que não ocorreu.
- Não gerar arquivos vazios.
- Não gerar funções sem implementação.
- Não gerar testes sem assertions relevantes.
- Não duplicar ações desnecessariamente.
- Não assumir que o SauceDemo executa pagamentos reais.
- Não esconder defeitos com retries.

- Não utilizar esperas fixas indiscriminadamente.
- Não armazenar segredos reais em fixtures.
- Não declarar execução aprovada sem evidências.
- Não adicionar arquitetura excessivamente complexa.
- Não gerar cenários exploratórios como testes funcionais aprovados sem oráculos confirmados.
- Não confundir quantidade de testes com cobertura efetiva.

5. Formato de saída (Output Format)

Entregue os resultados em blocos:

Bloco A — Análise da aplicação

Inventário funcional, regras, evidências e limitações.

Bloco B — Matriz RBT

Probabilidade, impacto, score e prioridade.

Bloco C — Plano de testes

Cenários, técnicas, assertions e classificação.

Bloco D — Inspeção de elementos

Mapeamento dos seletores e evidências.

Bloco E — Decisão arquitetural

Comparação entre Page Objects, App Actions e Custom Commands, com justificativa da escolha.

Bloco F — Estrutura do projeto

Árvore completa dos arquivos e responsabilidades.

Bloco G — Implementação

Código completo de todos os arquivos necessários.

Bloco H — Validação

Resultados de revisão estática, compilação e execução, quando disponíveis.

Bloco I — Métricas

Cobertura, duplicidade, aderência, cenários críticos omitidos, implementação e execução.

Bloco J — Entrega

Projeto ZIP, README, resumo executivo, pendências e instruções de utilização.

6. Critério final de qualidade

O resultado deve demonstrar um projeto de automação que possa ser compreendido, executado, mantido e ampliado por uma equipe de QA.

A arquitetura deve equilibrar:

- Qualidade das validações.
- Legibilidade.
- Reutilização.
- Desacoplamento.
- Estabilidade.
- Velocidade de diagnóstico.
- Custo de manutenção.
- Cobertura baseada em risco.

Não priorize volume de código. Priorize uma automação confiável e capaz de identificar defeitos relevantes.

Execute as etapas nesta ordem:

Exploração → RBT → Inspeção → Planejamento → Arquitetura → Implementação completa → Revisão → Execução → Métricas → Entrega.

2. Automação mobile com Robot Framework (inspeção de elementos e sintaxe)

Nesta etapa, **não é necessário configurar emulador ou executar o teste** (evitando custos e tempo de setup). O objetivo é avaliar sua habilidade com arquitetura de automação mobile e domínio de sintaxe com **Robot Framework + AppiumLibrary**.

Inspeção dos Elementos:

Você deve acessar o site **https://www.saucedemo.com/** pelo seu navegador, abrir as Ferramentas do Desenvolvedor (*DevTools / Inspecionar Elemento*) e inspecionar diretamente os IDs e atributos dos elementos que compõem o fluxo escolhido.

O que você deve entregar:

Defina o fluxo mobile mais relevante para o negócio e elabore um arquivo .robot contendo:

- Seção *** Settings *** com importação e configuração conceitual da AppiumLibrary.
- Seção *** Variables *** mapeando os localizadores que você identificou diretamente ao inspecionar o site.
- Seção *** Keywords *** demonstrando abstração, modularização, clareza semântica e reutilização de ações.
- Seção *** Test Cases *** descrevendo o fluxo selecionado de ponta a ponta.

Prompt — Projeto Completo de Automação Web Mobile | Robot Framework + AppiumLibrary | SauceDemo

1. Papel (Role)

Atue como um **QA Sênior / Quality Engineer especializado em automação mobile**, com experiência em:

- Robot Framework e AppiumLibrary.
- Automação web mobile em Android e iOS.
- Testes end-to-end de e-commerce.
- Inspeção de elementos com DevTools e Appium Inspector.
- Arquitetura modular de automação.
- Risk-Based Testing (RBT).
- Técnicas de testes ISTQB.
- Gerenciamento de massa de teste.
- Prevenção de flaky tests.
- Integração contínua e qualidade de código.

Sua responsabilidade é analisar a aplicação, planejar a cobertura e **implementar um projeto completo de automação**, pronto para ser aberto, compreendido, mantido e executado no VS Code.

Não entregue somente exemplos isolados. Gere os arquivos necessários para uma suíte

funcional, deixando explícitas as dependências e limitações.

2. Contexto (Context)

Aplicação: https://www.saucedemo.com/

Tipo: e-commerce demonstrativo acessado em navegador mobile.

Framework: Robot Framework.

Biblioteca de automação: AppiumLibrary.

Plataforma prioritária: Android com Chrome e Appium UiAutomator2.

IDE: Visual Studio Code.

Objetivo: criar uma suíte automatizada para os principais fluxos do SauceDemo, com cobertura orientada a riscos, arquitetura reutilizável e separação entre testes, keywords, seletores, dados e configuração.

O projeto deverá demonstrar práticas compatíveis com uma atuação de QA Sênior.

3. Instruções (Instruction)

Etapa 1 — Exploração da aplicação

Acesse o SauceDemo e investigue os seguintes módulos:

1. Autenticação e logout.
2. Catálogo de produtos.
3. Detalhes de produtos.
4. Ordenação por nome e preço.
5. Carrinho de compras.
6. Checkout e validações dos campos.
7. Resumo financeiro.
8. Finalização e confirmação.
9. Navegação e persistência de estado.
10. Comportamentos dos usuários especiais de demonstração.

Registre funcionalidades, regras observadas, dependências, limitações e evidências.

Não presuma funcionalidades que não foram verificadas.

Etapa 2 — Priorização por Risk-Based Testing

Identifique os riscos considerando:

- Impacto financeiro potencial em um e-commerce real.
- Impacto na conversão.
- Integridade dos produtos e preços.
- Autenticação e controle de acesso.
- Experiência do usuário.
- Probabilidade de falha.
- Criticidade para o negócio.

Utilize:

Score RBT = Probabilidade × Impacto

Ambos devem receber valores entre 1 e 5.

Classifique:

- **P0 — Crítico:** score de 16 a 25.
- **P1 — Alto:** score de 9 a 15.
- **P2 — Médio/Baixo:** score de 1 a 8.

A classificação deve ser justificada. Quando a probabilidade não estiver sustentada por histórico ou evidência, sinalize que se trata de estimativa.

Separe cenários de smoke, regressão essencial, regressão complementar e testes exploratórios.

Etapa 3 — Inspeção dos elementos

Abra a aplicação no navegador e utilize DevTools para inspecionar diretamente os elementos dos fluxos selecionados.

Identifique:

- Tag HTML.
- ID.
- Atributos `data-test`, `data-testid` ou equivalentes.
- Atributos de acessibilidade.
- Seletor CSS ou XPath.
- Estado e comportamento do elemento.
- Evidência utilizada.

Priorize seletores estáveis e semanticamente significativos.

Não invente localizadores.

Quando a inspeção direta não estiver disponível, consulte fontes confiáveis, identifique os seletores como provisórios e registre a necessidade de validação.

Confirme a compatibilidade dos seletores com o navegador mobile, sempre que houver

ambiente disponível.

Etapa 4 — Arquitetura do projeto

Crie obrigatoriamente a seguinte estrutura:

saucedemo-robot-mobile/

```
│
├── tests/
│   ├── 01_auth.robot
│   ├── 02_catalog.robot
│   ├── 03_cart.robot
│   ├── 04_checkout.robot
│   ├── 05_checkout_validation.robot
│   └── 06_exploratory.robot
│
├── resources/
│   ├── keywords/
│   │   ├── browser.robot
│   │   ├── auth.robot
│   │   ├── catalog.robot
│   │   ├── cart.robot
│   │   └── checkout.robot
│   │
│   └── locators/
│       └── saucedemo.robot
│
├── variables/
```

```
│   ├── config.py
│   └── testdata.robot
│
├── results/
│
├── requirements.txt
├── README.md
└── .gitignore
```

Todos os arquivos devem conter conteúdo adequado às suas responsabilidades. Não crie arquivos vazios apenas para representar a arquitetura.

O diretório `results/` pode iniciar vazio, pois será preenchido durante a execução.

Etapa 5 — Geração completa dos Test Cases

Implemente os cenários automatizáveis identificados no planejamento RBT.

A cobertura inicial deve considerar, no mínimo, os seguintes candidatos, sujeitos à confirmação dos requisitos:

Arquivo `01_auth.robot`

- TC01 — Login com usuário válido.
- TC02 — Bloqueio de usuário impedido de autenticar.
- TC03 — Tentativa de acesso a página protegida sem autenticação.
- TC15 — Login com credenciais inválidas.
- TC16 — Logout e tentativa de acesso após encerramento da sessão.

Arquivo `02_catalog.robot`

- TC17 — Ordenação de produtos por preço crescente.
- TC18 — Ordenação por preço decrescente.
- TC19 — Consistência entre catálogo e detalhes do produto.
- TC21 — Ordenação por nome A–Z e Z–A, preferencialmente com casos independentes ou parametrizados.

Arquivo `03_cart.robot`

- TC04 — Adicionar produto ao carrinho.
- TC05 — Adicionar múltiplos produtos.
- TC10 — Remover produto.

- TC11 — Preservação dos itens durante navegação.

Arquivo `04_checkout.robot`

- TC06 — Compra completa com um produto.
- TC07 — Compra completa com múltiplos produtos.
- TC08 — Validação de subtotal, imposto e total.
- TC09 — Consistência dos produtos no resumo.
- TC20 — Cancelamento e navegação durante checkout.

Arquivo `05_checkout_validation.robot`

- TC12 — Primeiro nome obrigatório.
- TC13 — Sobrenome obrigatório.
- TC14 — Código postal obrigatório.

Arquivo `06_exploratory.robot`

Documente inicialmente:

- TC22 — Integridade visual e apresentação de imagens.
- TC23 — Avaliação de lentidão com usuário especial.
- TC24 — Investigação de checkout com carrinho vazio.

Não transforme os cenários exploratórios em assertions definitivas sem comportamento esperado confirmado.

Caso algum dos 21 candidatos automatizáveis não possa ser implementado de forma confiável, registre o motivo, a dependência e o status pendente. Não substitua testes ausentes por implementações vazias que aparentem sucesso.

Etapa 6 — Implementação das Keywords

Implemente todas as keywords chamadas pelos Test Cases.

Organize as responsabilidades por módulo.

`browser.robot`

- Abrir navegador mobile.
- Configurar sessão Appium.
- Navegar para a aplicação.
- Encerrar sessão.
- Preparar o estado inicial.

`auth.robot`

- Preencher credenciais.
- Executar login.

- Validar autenticação.
- Validar mensagem de erro.
- Executar logout.

`catalog.robot`

- Identificar produtos.
- Selecionar produto.
- Verificar dados.
- Alterar ordenação.
- Validar lista ordenada.

`cart.robot`

- Adicionar produto.
- Remover produto.
- Abrir carrinho.
- Validar quantidade.
- Validar nome e preço.
- Validar preservação de estado.

`checkout.robot`

- Iniciar checkout.
- Preencher informações.
- Validar campos obrigatórios.
- Avançar para o resumo.
- Validar itens e valores.
- Finalizar a compra.
- Validar confirmação.

Utilize `[Arguments]` para receber dados e `RETURN` quando houver necessidade de devolver resultados.

Evite duplicação entre keywords.

Não crie keywords genéricas demais que escondam toda a lógica de negócio ou tornem o diagnóstico de falhas difícil.

Etapa 7 — Localizadores desacoplados

Centralize os localizadores em:

`resources/locators/saucedemo.robot`

Utilize nomes semânticos, por exemplo:

${LOGIN_USERNAME}

${LOGIN_PASSWORD}

${LOGIN_BUTTON}

${LOGIN_ERROR}

${PRODUCT_LIST}

${CART_LINK}

${CHECKOUT_BUTTON}

${CHECKOUT_TOTAL}

Os nomes acima representam convenções; os valores dos localizadores devem vir da inspeção ou de referências identificadas como provisórias.

Não duplique seletores em vários Test Cases.

Para elementos dinâmicos, avalie a utilização de keywords parametrizadas ou localizadores construídos a partir de atributos estáveis confirmados.

Etapa 8 — Separação de massa de teste e configuração

Utilize:

`variables/config.py`

Centralize configurações de ambiente:

- URL da aplicação.
- Endereço do Appium Server.
- Plataforma.
- Navegador.
- Identificador do dispositivo ou emulador, quando necessário.
- Timeouts.
- Configurações não sensíveis de execução.

Permita sobrescrever configurações conforme o ambiente.

`variables/testdata.robot`

Defina:

- Usuários de demonstração.
- Dados válidos de checkout.
- Dados inválidos.
- Produtos de referência.
- Preços esperados.

- Mensagens esperadas.
- Valores de referência sustentados por evidências.

Diferencie dados de teste de regras de negócio.

Nunca armazene credenciais privadas em arquivos versionados.

Etapa 9 — Assertions e oráculos

Cada Test Case deve possuir validações suficientes para comprovar seu objetivo.

Não considere uma ação bem-sucedida apenas porque o elemento foi clicado.

Inclua, quando aplicável:

- URL ou tela esperada.
- Estado autenticado.
- Mensagem de erro correta.
- Nome do produto.
- Preço.
- Quantidade.
- Identidade dos itens do carrinho.
- Subtotal.
- Imposto.
- Total.
- Confirmação da finalização.

Para valores financeiros, utilize cálculos independentes fundamentados nas regras verificadas.

Evite assertions que apenas comparam dois valores produzidos pela própria aplicação sem uma referência confiável.

Não confunda ausência de falha técnica com comportamento correto.

Etapa 10 — Prevenção de flaky tests

Implemente:

- Esperas condicionais.
- Elementos localizados por atributos estáveis.
- Preparação independente por Test Case.
- Setup e teardown consistentes.
- Limpeza de estado.
- Ausência de dependência entre testes.
- Logs e screenshots em caso de falha.
- Tratamento de operações assíncronas.
- Timeouts configuráveis.

Evite `Sleep` fixo, cliques em posições absolutas e retries indiscriminados.

Explique os pontos de maior risco de instabilidade.

Etapa 11 — Arquivos de suporte

`requirements.txt`

Liste as dependências Python necessárias à instalação, incluindo Robot Framework e AppiumLibrary.

Utilize versões compatíveis, quando possível, e documente como reproduzir o ambiente.

`README.md`

Inclua:

1. Objetivo do projeto.
2. Arquitetura de pastas.
3. Pré-requisitos.
4. Configuração do Python.
5. Instalação das bibliotecas.
6. Configuração do Android SDK.
7. Instalação do Appium Server e UiAutomator2.
8. Preparação do emulador ou dispositivo.
9. Comandos de execução.
10. Execução por suíte.
11. Execução por tags.
12. Localização de logs, relatórios e screenshots.
13. Convenções de desenvolvimento.
14. Limitações e pendências conhecidas.

Inclua comandos para Windows PowerShell, considerando uso no VS Code.

`.gitignore`

Ignore ambientes virtuais, caches, logs, relatórios, screenshots gerados, arquivos temporários e dados sensíveis locais.

Etapa 12 — Revisão de integridade do projeto

Antes de entregar, confira:

- Todos os imports de recursos.
- Caminhos relativos.
- Keywords chamadas e implementadas.
- Variáveis declaradas e utilizadas.
- Dependências necessárias.
- Sintaxe Robot Framework.

- Compatibilidade com AppiumLibrary.
- Uso correto de `${variavel}`, `@{lista}` e `&{dicionario}`.
- Assertions e dados esperados.
- Ausência de Test Cases vazios.
- Ausência de keywords fictícias sem implementação.
- Independência e isolamento.
- Clareza dos nomes.
- Priorização por tags.

Quando o ambiente permitir, execute:

robot --dryrun -d results tests

Depois execute os testes de smoke em dispositivo ou emulador.

Caso não seja possível instalar bibliotecas ou executar o Appium, informe claramente que a validação ficou pendente.

Não declare testes aprovados sem execução real.

Etapa 13 — Entrega do projeto

Gere fisicamente todos os arquivos definidos na arquitetura.

Não apresente apenas a árvore de diretórios ou fragmentos de código.

Crie um pacote ZIP contendo a estrutura completa:

`saucedemo-robot-mobile-completo.zip`

O arquivo deve permitir que o usuário:

1. Extraia o ZIP no Windows.
2. Abra a pasta no VS Code.
3. Instale as dependências.
4. Configure o Appium e o dispositivo.
5. Execute o `--dryrun`.
6. Execute os testes automatizados elegíveis.
7. Consulte os relatórios de execução.

Forneça também o `README.md` individualmente para consulta rápida.

Se o ambiente não permitir criar arquivos, entregue o conteúdo completo de cada arquivo, com seu respectivo caminho, sem afirmar que gerou um ZIP.

4. Restrições (Constraints)

- Não inventar funcionalidades.

- Não inventar seletores ou endpoints.
- Não afirmar inspeção realizada sem evidência.
- Não considerar SauceDemo um aplicativo nativo.
- Não utilizar SeleniumLibrary em substituição à AppiumLibrary.
- Não entregar somente um arquivo `.robot`.
- Não criar testes sem assertions relevantes.
- Não criar keywords sem implementação.
- Não gerar código duplicado desnecessariamente.
- Não usar esperas fixas como solução padrão.
- Não considerar cenários exploratórios como automaticamente aprovados.
- Não afirmar que o checkout envolve transação financeira real.
- Não declarar cobertura total sem requisitos rastreáveis.
- Não declarar execução bem-sucedida sem logs.

5. Formato de saída (Output Format)

Entregue os seguintes blocos:

Bloco A — Mapeamento funcional

Funcionalidades, regras e dependências identificadas.

Bloco B — Matriz de riscos

Probabilidade, impacto, score RBT, prioridade e justificativa.

Bloco C — Plano de testes

Todos os cenários selecionados, técnicas de teste, assertions esperadas e prioridades.

Bloco D — Mapeamento de elementos

Seletores, atributos, estratégia de localização, evidência e status de confirmação.

Bloco E — Arquitetura

Árvore de pastas e responsabilidades de cada arquivo.

Bloco F — Implementação

Todos os arquivos `.robot`, `.py`, `.txt`, `.md` e demais configurações necessários.

Bloco G — Validação técnica

Resultado da revisão estática, `--dryrun`, execução real quando disponível e pendências.

Bloco H — Métricas de qualidade

Apresente:

- Cobertura de requisitos.
- Taxa de duplicidade.
- Aderência à especificação.
- Cenários críticos omitidos.
- Cenários implementados versus planejados.
- Cenários executados e aprovados, quando houver execução real.

Apresente fórmulas, numeradores, denominadores e limitações da análise.

Bloco I — Entrega dos artefatos

Disponibilize:

- Projeto completo em ZIP.
- README individual.
- Resumo dos cenários implementados.
- Lista de pendências.
- Instruções de execução.

6. Critério final de qualidade

A solução deve demonstrar uma arquitetura de automação sustentável, tecnicamente consistente e orientada a risco.

O projeto precisa ser compreensível para outros QAs, permitir a execução independente dos testes e facilitar investigação de falhas e evolução da cobertura.

Não priorize quantidade de código em detrimento da qualidade.

Siga a sequência:

Exploração → RBT → Inspeção → Planejamento → Arquitetura → Geração completa dos arquivos → Revisão → Execução → Entrega.

3. Engenharia de performance, métricas e diagnóstico de falhas

Como QA Sênior em um e-commerce, você é o ponto focal para assegurar que a plataforma

suporte grandes eventos sazonais (ex.: Black Friday, campanhas relâmpago).

Responda detalhadamente com visão estratégica e técnica:

1. **Estratégia e Tipos de Testes:**
    - Quais testes de performance você desenharia para o fluxo de compra e navegação do e-commerce (ex.: Carga, Estresse, Pico/Spike, Resistência/Soak, Capacidade Máxima)?

    - Qual o objetivo de cada um e qual a volumetria/perfil de usuário recomendaria simular?

2. **Ferramental, Métricas e Metrificação:**
    - Quais métricas de aplicação (Latência p95/p99, Throughput/RPS, Taxa de Erro 4xx/5xx) e de infraestrutura (CPU, Memória, IOPS, Network I/O) seriam acompanhadas?

    - Como você estabelece os critérios de aceite (SLIs / SLOs) para aprovar ou reprovar uma versão?

3. **Troubleshooting e Melhoria de Código/Arquitetura:**
    - Diante de um aumento crítico de latência no fechamento do pedido sob carga elevada, descreva seu passo a passo investigativo.

    - Quais recomendações técnicas você discutiria com engenheiros de backend, infraestrutura e banco de dados para eliminar os gargalos (ex.: caching com Redis, otimização de queries, filas assíncronas, tuning de pool de conexões)?

4. Inteligência Artificial no ciclo de vida da qualidade

Com a maturidade esperada de um perfil Sênior, descreva como aplicar Inteligência Artificial de forma prática, escalável e responsável:

- **Geração e cobertura de cenários:** Como a IA pode ser utilizada para descobrir casos de borda (*edge cases*) e falhas de lógica de negócio que comumente passam despercebidas no levantamento manual?

- **Produtividade sem perda de qualidade:** Como acelerar a criação e manutenção de scripts (Cypress, Robot e k6/JMeter) evitando a proliferação de código descartável, alucinações ou testes instáveis (*flaky tests*)?

- **Análise preditiva e observabilidade:** Como ferramentas de IA podem apoiar na análise de logs, agrupamento de falhas em esteiras de CI/CD e diagnóstico antecipado de anomalias de performance?